

Leonardo Corso

DBGuard

User manual

Leonardo Corso
5-11-2026

Contents

What is DBGuard?	3
Getting started.....	4
Installation	4
Guards	6
Understanding a guard	6
Guard database connection.....	6
Create a guard.....	6
Example guard	9
View guards.....	12
Guard change history	13
Understanding guard changes	13
View guard changes.....	13
Database connections.....	13
Understanding a database connection	13
Create a database connection	14
Examples.....	15
Local database server.....	15
Remote database server.....	16
Local database file	16
View database connections	16
Notifications.....	17
Notification providers	17
Understanding notification providers	17
Create a notification provider	17
Guard notifications.....	19
Understanding notifications	19
Create notifications	19
View notifications	22
Notification transactions	22
Users.....	23

User info	23
Create users	23
View users	24
Miscellaneous	24
Dashboard	24
Settings.....	24

What is DBGuard?

DBGuard is an application that periodically checks the state of your databases and sends notifications when specific conditions are met. It is useful for catching bad or unexpected data that can cause applications to crash, or for alerting a team when data meets any criteria that warrants attention.

DBGuard supports multiple database engines including SQL Server, PostgreSQL, SQLite, and MySQL.

Monitoring is performed through Guards — each Guard runs a user-defined SQL query on a periodic schedule and compares the result to a user-defined expression. Since the core logic and business rules are written in SQL, the application remains decoupled and simple to configure.

When a Guard's expression evaluates to true, it can trigger zero or more notifications via SMTP (email) or HTTP (webhooks to other applications or APIs). DBGuard also includes a built-in SPA web application for configuring, testing, and auditing Guards and their execution history.

Getting started

Installation

The recommended installation method is through docker. A simple configurable image is available for DBGuard

(<https://hub.docker.com/repository/docker/leonardocodes/dbguard>) and you can use the docker compose file below to be up and running:

services:

postgres:

image: postgres:latest

restart: unless-stopped

environment:

POSTGRES_DB: dbguard-web

POSTGRES_USER: dbguard-web

POSTGRES_PASSWORD: abc123##

volumes:

- postgres_data:/var/lib/postgresql

ports:

- "5432:5432"

healthcheck:

test: ["CMD-SHELL", "pg_isready -U dbguard-web -d dbguard-web"]

interval: 5s

timeout: 5s

retries: 5

dbguard:

image: dbguard:latest

restart: unless-stopped

ports:

- "8080:8080"

volumes:

- app_data:/app/data

depends_on:

postgres:

condition: service_healthy

environment:

ASPNETCORE_ENVIRONMENT: Production

ASPNETCORE_URLS: http://+:8080

APP_HOST: 10.55.47.124

APP_PORT: 8080

#AppDataPath: # Default is /app/data)

#DATABASE_HOST: # Default is postgres (Used to overwrite connection string host)

#DATABASE_PORT: # Default is 5432

#DATABASE_NAME: # Default is dbguard-web

#DATABASE_USER: # Default is dbguard-web

#DATABASE_PASSWORD: # Required if settings custom
POSTGRES_PASSWORD value default is abc123##

#JWT_KEY: # Default is generated and stored in file (Used to issue tokens)

#JWT_ISSUER: # Default is generated from APP_HOST and APP_PORT (Used in token claims)

#JWT_AUDIENCE: # Default is generated from APP_HOST and APP_PORT (Used in token claims)

#JWT_ACCESS_TOKEN_EXPIRATION: # Default is 15 minutes

#JWT_REFRESH_TOKEN_EXPIRATION: # Default is 1440 minutes

#APP_USERNAME: # Default is admin (The default admin user bootstrapped)

#PASSWORD: # Default is Admin123!

#ALLOWED_ORIGINS: # Default is built from APP_HOST and APP_PORT (Should be list of hosts http:10.55.47.123:4200,http:10.55.47.124:4201, ..)

volumes:

postgres_data:

app_data:

Guards

Understanding a guard

A guard is a single query against a single database that evaluates the result to a user defined expression. The guard is created and run at specific intervals as some guards may only need to run once a week, while others multiple times an hour. A guard can be in one of four states:

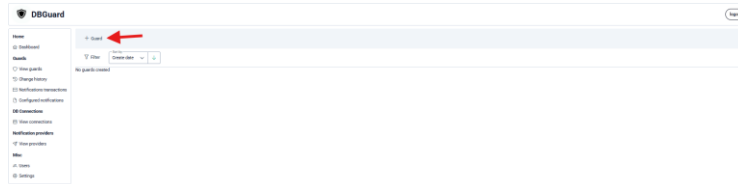
- **Unknown:** The initial state of the guard. Indicates the scan has not yet run.
- **Error:** The guard ran into one of various errors such as it could not connect to the database, the query is invalid, the result column it's defined to look at does not exist in the result set, or any other error which can occur when a query is ran from a local or remote application against a database.
- **Triggered:** The user defined query and expression are evaluated as true.
- **Clear:** The user defined query and expression is evaluated as false.

Guard database connection

A database connection is used by the guard to connect to a database. The database connection can be reused by many guards. If the database engine requires a user login, the database user should have only read permission to the database to ensure best security practices. The credentials are encrypted and stored in the PostgreSQL application database using an IDataProtector. Some database engines such as SQLite do not use logins so it is not required.

Create a guard

1. To create a guard, go to "/guards/view-all" or select "View guards" from the sidebar menu.
2. Select the "+Guard" button on the toolbar:



3. You will then see the dialog to create the guard:

Create guard



Is active



Guard name

Guard description

Trigger query

Required

Trigger column

Required

Trigger Operator

Required

Trigger value

Required

Database connection

Required

Run period (minutes)
5

Required

Notifications

No notifications added



Notification provider

Notify on clear Notify on error Notify on trigger

Validate guard: ☒



4. Here is what each field does:

- **Is active:** Boolean that determines if the guard is active or paused and should be run by the monitoring service.
- **Guard name:** Optional user-friendly name.
- **Guard description:** Optional description of the guard.
- **Trigger query:** The SQL query for the guard. The query needs to return a single row. The reason for this is that all the logic should be implemented in the query and we only need a single record so we can look at the trigger column to evaluate the expression.
- **Trigger column:** This is the name of the column in our queries result record that is used in the expression. Its value should be a 32-bit integer as it will be used in the expression to evaluate the guard state.
- **Trigger operator:** The operator for the expression. Supports equals, not equals, greater than, less than, greater than or equal to, less than or equal to.
- **Trigger value:** A 32-bit integer value for our expression.
- **Database connection:** A connection and login if required to our database.
- **Run period:** How often in minutes the guard will be run.
- **Notifications:** The notifications for the guard.
- **Notify on clear, error, trigger:** Whether the guard should notify using its configured notifications when it goes into the clear, error, or triggered state respectively.
- **Validate guard:** Whether to validate the guard before creation to ensure it can connect to the database, the query is valid and trigger column exists in the result set. Will return an error with failure if it is invalid.

5. Click add.

Example guard

For example, let's say we want to be notified once a day if there are any customers in our database who have a debt that is more than \$10.50. This is a good example to show that although the guard trigger column value is a 32-bit integer, it can support monitoring decimals, dates, strings and many other data types since the logic is stored in the SQL query, and the application simply checks a trigger value.

1. Our trigger query may look something like this:

```
SELECT COUNT(*) as HighDebt
FROM customer_debt
WHERE customer_debt.dollars > 10.5;
```

2. The column we want to look at is called "*HighDebt*" so the trigger column will be "HighDebt".
3. Since the query uses ">" our trigger operator will be greater than or equal to.
4. Lastly the trigger value will be 1 since we want to be notified if any exist.
5. The expression will then read as, trigger guard when "HighDebt" column in query result is greater than 1.
6. Since we want it to run once a day, we will set the run period in minutes to 1440, so it runs every 24 hours.

Create guard



Is active



Guard name

High Debt Example

Guard description

Demo guard to showcase a high debt

Trigger query

```
SELECT COUNT(*) AS HighDebt
FROM customer_debt
WHERE customer_debt.debt_dollars > 10.5;
```

Required

Trigger column

HighDebt

Required

Trigger Operator

Equals

Required

Trigger value

1

Required

Database connection

- Database type: PostgreSQL
- Endpoint: [REDACTED]
- Database name: [REDACTED]
- Username: [REDACTED]



Required

Run period (minutes)

1,440

Required

Notifications



No notifications added



Notification provider



Notify on clear



Notify on error



Notify on trigger



Validate guard: ☒

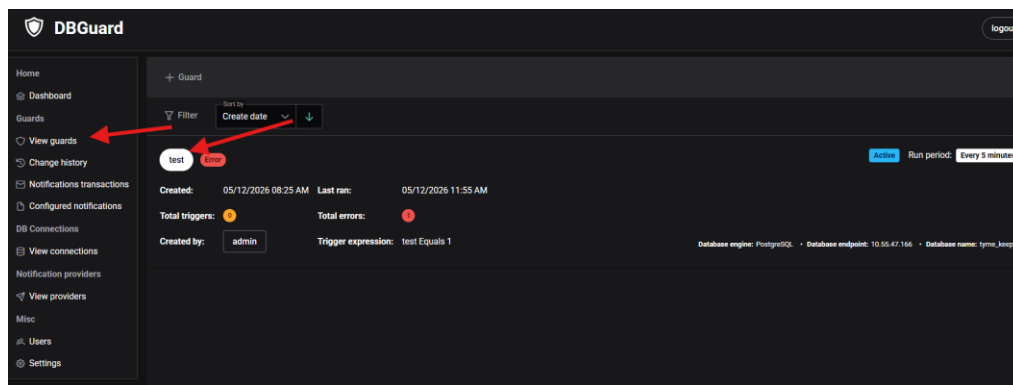
Add

View guards

You can view the created guards by selecting the “View guards” navigation on the sidebar or navigating to “/guards/view-all”.

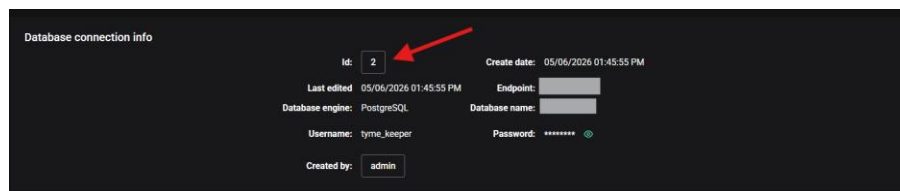
The header includes a sort by selection and a filter button to filter and sort your guards. Additionally, you can use the paginator on the bottom to change the view count.

The view-all webpage displays a summary of the most important information for the guard. You can select the guard’s name to view its details.

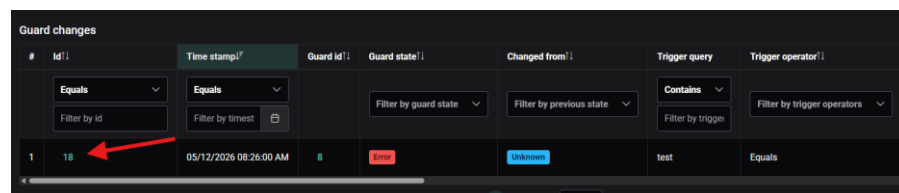


Selecting the guard’s name link navigates you to “/guards/detail/:id”. It displays The details of the guard, its database connection, its configured notifications, the guard state change history, and guard notification transactions sent. Many of the properties are links which can be used to navigate to other detail pages:

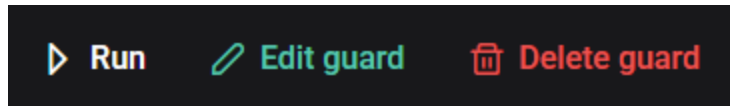
Link to database connection detail:



Link to guard change detail:



From the details page you can also manually run a guard using the “Run” button. Admins can also edit or delete guards here:



Guard change history

Understanding guard changes

When a guard state changes, a log is created for auditing purposes. The guard details at the time of change are stored in addition to the state it changed from and changed to. It also stores the result of the trigger column which caused the state change.

View guard changes

You can view guard changes by clicking the “Change history” navigation on the sidebar or navigating to “/guards/history-detail”.

This webpage displays a table of the guard state changes where you can sort, filter and navigate the paginated changes.

Clicking on the change id will navigate you to the change details at “/guards/history-detail/:id”.

Guard change history				
#	Id↑↓	Time stamp↑↓	Guard id↑↓	Guard state↑↓
	Equals Filter by id	Equals Filter by timesta	Equals Filter by guard id	Filter by guard state
1	18	05/12/2026 08:26:00 AM	8	Error
2	17	05/08/2026 09:56:20 AM	DELETED	Error

The change details will show a lot more information for the guard state change. Additionally, it will show the notifications that were sent out when that change occurred.

If the guard is not deleted it will also show the guards current information.

Database connections

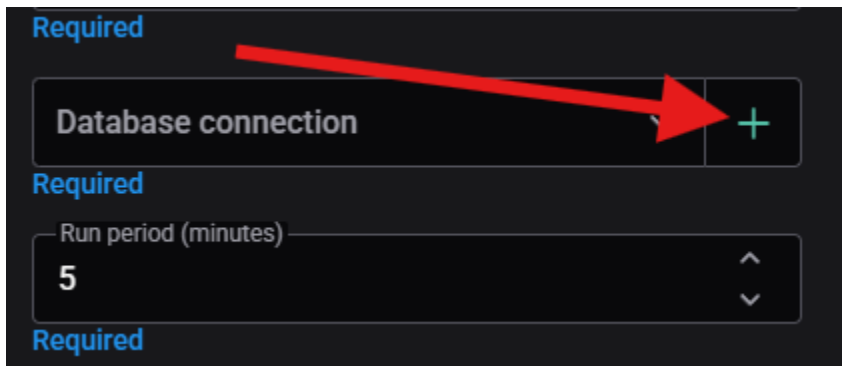
Understanding a database connection

A database connection is used by the guard to connect to the database. You can share the database connections between multiple guards. The guards should have read-only permissions to the database, schema, or table.

Some database engines such as SQLite do not use servers or user logins. For these, you instead specify the path of the database and can leave the username and password empty.

Create a database connection

1. You can create a database connection by selecting the + button in the guard creation dialog:



2. Alternatively, you can navigate to “/db-connections/view-all” or select “View connections” from the sidebar.
3. You will see the creation dialog appear:

Create Database Connection

Required

Required

Required

Validate connection: ☒

Add

4. Here is what each of the fields mean:

- **Endpoint:** The hostname, IP Address or system path to the server or database file
- **Database Engine:** The database engine type.
- **Database name:** The database or database file name.
- **Username:** The database username if applicable.
- **Password:** The database password if applicable.
- **Validate connection:** Will validate the service can connect to the database server or can find the database file.

Examples

Local database server

- Endpoint: 10.55.48.112
- Database Engine: PostgreSQL

- Database Name: Demo
- Username: read-only
- Password: Password1

Remote database server

- Endpoint: ssms.website.net
- Database Engine: SQL Server
- Database Name: Demo
- Username: read-only
- Password: Password1

Local database file

- Endpoint: C:\Users\Leo\Downloads\database.db
- Database: SQLite
- Database Name: database.db

View database connections

You can view the database connections by clicking the “View connections” navigation on the sidebar. Alternatively, you can navigate to “/db-connections/view-all”.

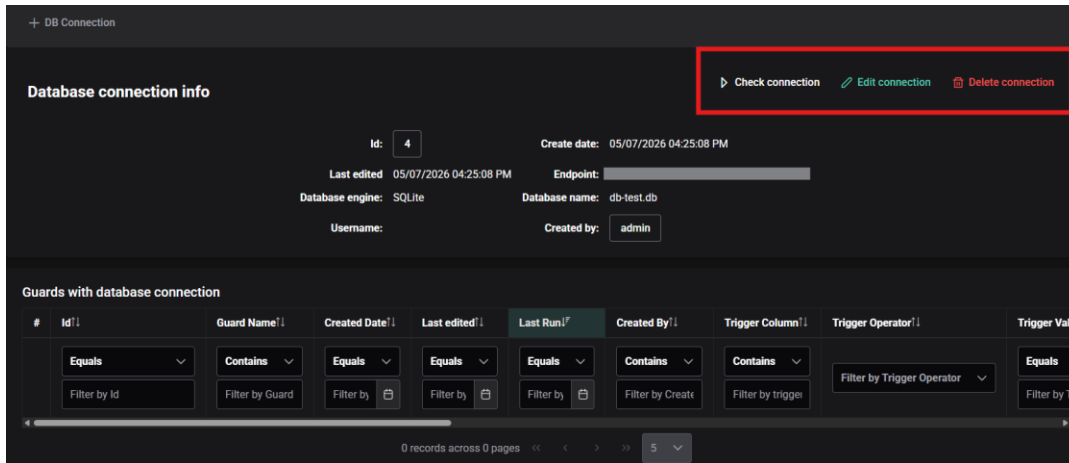
This webpage will display a table with the database connection details. You can filter, sort and view the connection pages.

To view the details of the connection, you can click on the id link.

#	Id	Endpoint	Database engine	Username	Create date	Last edited	Created by
1	4		SQLite		05/07/2026 04:25:08 PM	05/07/2026 04:25:08 PM	admin
2	3		SQLServer	test	05/07/2026 04:22:18 PM	05/07/2026 04:22:18 PM	admin
3	2		PostgreSQL	tyrne_keeper	05/06/2026 01:45:55 PM	05/06/2026 01:45:55 PM	admin

The details can also be navigated to at “/db-connections/detail/:id”. It will display the database connection details, and the guards using this database connection.

From this page you can also test the database connection and administrators can edit or delete the connection.



Notifications

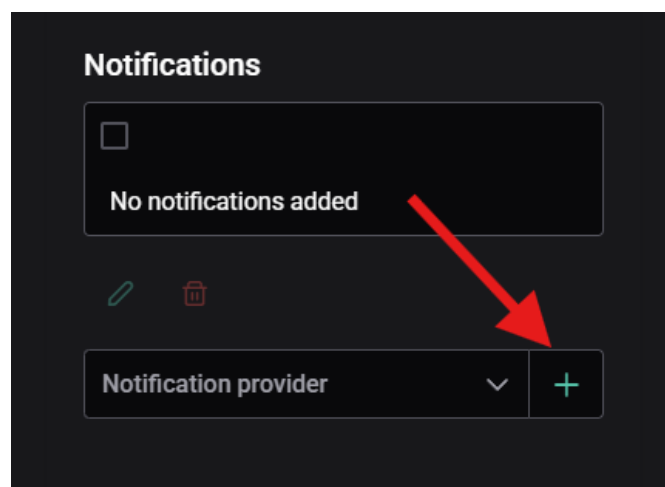
Notification providers

Understanding notification providers

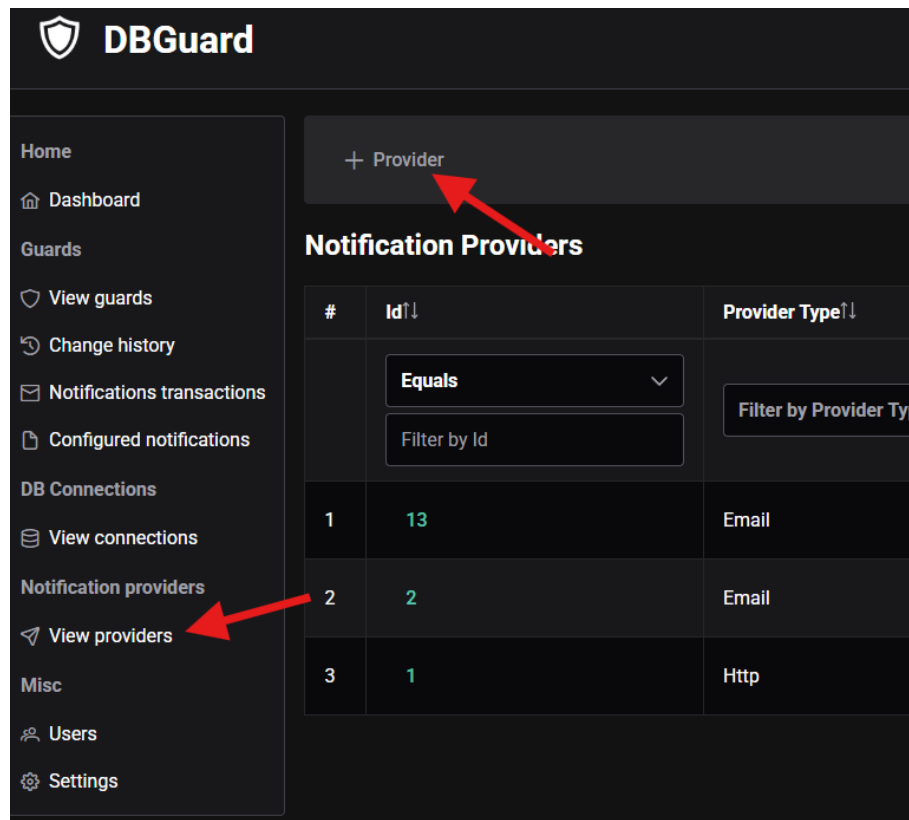
Notification providers are used by notifications to send out their alerts. Two types of providers are supported, Email providers and HTTP providers. Email providers are created by the user as it requires SMTP information. HTTP providers on the other hand can be used directly in a notification as it simply requires a URL.

Create a notification provider

1. You can create a notification provider by selecting the + button from the guard creation screen:



2. Alternatively, you can select on "View providers" from the sidebar and the select the +Provider button on the tool bar:



3. Select the provider type (Note that only email providers can be configured).
4. These are the meanings of each field:
 - Provider type: The type of provider you're configuring. Only email is supported for configuring.
 - SMTP Server: The server or hostname of your SMTP service.
 - Port: The SMTP server port (Default is 587).
 - Username: The SMTP service username.
 - Password: The SMTP service password.
 - Sender email: The sender email for the notifications. Usually this is the same as username (Not always the case).
 - Verify provider: Will try to login to the SMTP service to ensure information is correct.

Create notification provider ✕

Provider type
Email

SMTP Server
smtp.service.com
Required

Port
587
Required

Username
dotnotreply@email.com
Required

Password
.....
Required

Sender email
dotnotreply@email.com
Required

Verify provider: ☒

+ Add

Guard notifications

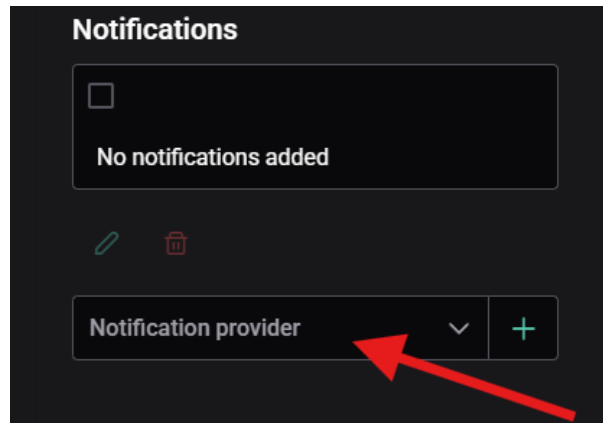
Understanding notifications

A notification is added to a guard and gets processed when the guards state changes (If configured). DBGuard currently supports email notifications via SMTP or HTTP webhooks. Each notification uses a notification provider that includes the SMTP information or the HTTP provider config.

Create notifications

You can create a notification from the guard creation dialog.

1. Start by selecting the notification provider.

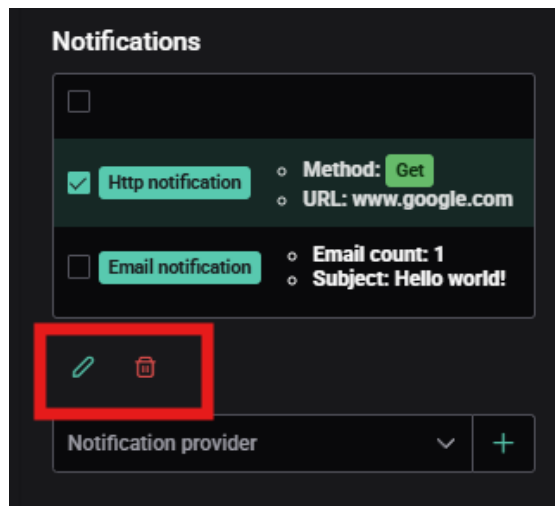


2. Depending on the provider selected, you will see a form to configure SMTP notifications, or HTTP webhooks.
 - HTTP notification form:

A screenshot of the 'HTTP notification form' in a dark-themed application. The form is titled 'Notifications'. It starts with a 'No notifications added' message. Below that are two small icons: a pencil and a trash can. The 'Notification provider' dropdown is set to 'Http webhook'. Below this is a 'URL' field with a 'Required' label and a 'Body type' dropdown. The 'Request headers' section has a 'No request headers added' message and a table with 'Header key' and 'Header value' columns, and an 'Add header' button. The 'Query parameters' section has a 'No query parameters added' message and a table with 'Parameter key' and 'Parameter value' columns, and an 'Add parameter' button. At the bottom, there are 'Add' and 'Cancel' buttons.

- Email notification form:

- The following can be configured for each notification type:
 - Email notification:
 - TO Emails: List of emails to send as to recipients.
 - CC Emails: List of emails to send as cc recipients.
 - BCC Emails: List of emails to send as bcc recipients.
 - Email subject: The subject of the email notification.
 - Email body: A body message to add to the email notification
 - HTTP notification:
 - URL: The URL to send the HTTP request.
 - Body type: The form body type if sending form data (Form data, Raw, Form URL Encoded).
 - Body data: Input for body data (Will display after selecting body type).
 - Place holders: Data placeholders which can be dynamically embedded into body when HTTP notification is processed and sent.
 - Request headers: Request headers to add into HTTP request (Useful for API keys or other required headers).
 - Query parameters: Additional parameters needed for the request.
3. Once your notification info is input, click the Add button.
 4. You will see the notifications listed. You can edit or delete a notification by selecting it, then clicking the edit or save icon.



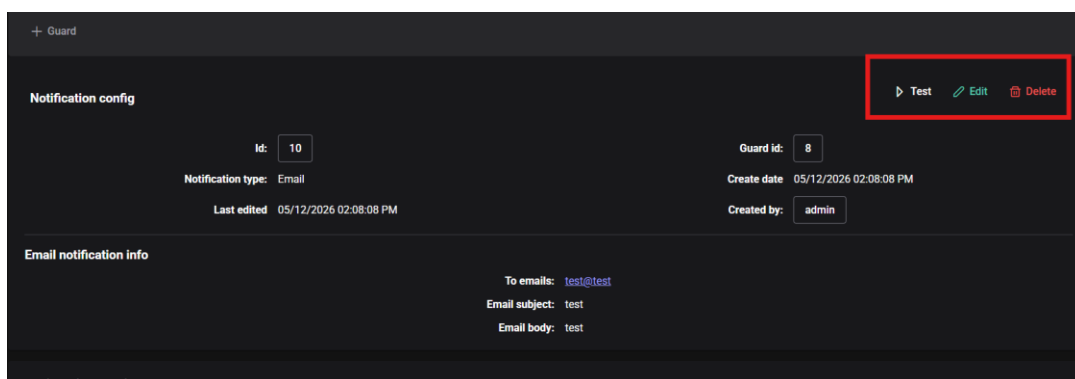
View notifications

You can view configured notifications by selecting the “Configured notifications” navigation in the sidebar menu or by going to `/guards/configured-notifications`. This will display a list of configured notifications that can be sorted, filtered and paginated.

To view more details on the notification you can click on the id link. Clicking the link will navigate you to the detail’s component located at `/guards/notification-config-detail/:id`.

The webpage will display the notification info, the provider details, the history of the notification transactions and the guard information for the notification.

You can also test the notification by selecting “Test” in the top header. Admins can also edit and delete guard notifications here.



Notification transactions

Understanding notification transactions

A notification transaction is a log of the notification that was sent when the guard state changed. It allows you to view any notifications that were sent or failed to send. It

records the notification information at the time it was processed as well as any responses in the case of HTTP notification.

View notification transactions

You can view notification transaction by selecting “Notification transactions” from the sidebar menu. You can also navigate to “/guards/notification-transactions”.

The webpage will display a table of notification transactions which can be sorted, filtered and paginated.

You can select the transaction id link from the table to view the transaction details. This will navigate you to the details page at “/guards/notifications-transaction-detail/:id”.

This webpage will display the transaction information, and specific details for the type of transaction that was sent (Email or HTTP). It also will display the guard change information which triggered the notification.

Users

User info

Users contain a unique username and a password. The password must be 8 characters minimum, include one lowercase and one uppercase character, at least one digit and one special character. Users can be inactivated if needed.

The application uses two roles for users to limit actions. The two roles are “User” and “Admin”. Users can generally view the data in the application (excluding database passwords). Admins on the other hand can modify entities (Guards, connections, notifications, users).

A default admin user is created when the application is first run. This user should only be used for initial configuration and subsequently deleted for best security practices. The “admin” username is reserved and cannot be used for new users.

Create users

You can create a user by clicking the “Users” navigation link in the sidebar. You can also navigate to “/users/view-all”.

1. Select the +User button on the toolbar and the creation dialog will appear.
2. User fields:
 - **Username:** The unique username.
 - **Password:** The password for login.
 - **Confirm password:** Confirm password.
 - **Roles:** The roles for the user (User, Admin).

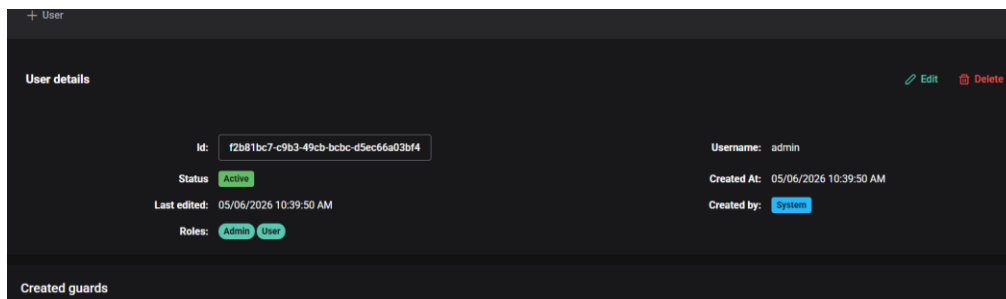
- **Is active:** Whether the user is active.
3. Click create.

View users

To view the users, you will need to login using an administrator account. You can select “Users” from the sidebar menu. This will display a table of users which can be sorted, filtered, and paginated.

To view a user’s details, you can click the id link from the table. This will navigate you to “/users/detail/:id”. The webpage will display the users’ details, the guards created by the user, the database connections created by the user and notification providers the user created.

On the top header, you can also edit or delete the user. Do note you can also deactivate them instead of fully deleting them from the system.



Miscellaneous

Dashboard

The dashboard is the landing page of the application. It lists a totals summary displaying the number of guards, notifications, providers, database connections, users and guard changes.

It also displays a helpful chart with the guard state changes for each month of the current year.

Settings

The settings webpage lists the current user’s details and allows updating their username or password.

The user can also toggle light and dark modes here.